Queries, Criteria API, and JPQL

Chapter 13

Introduction

- How do we find entity beans?
- A primary key is required
- Usually will want to find entity bean(s) using some criteria
  - E.g. find all employees with a given name
- EJB standard has a query language for defining custom *queries*
- JPQL defined on abstract persistence schema of entity beans
    - Not on underlying data store
  - references entity properties and relationships

Query API

- `jakarta.persistence.Query` is an interface
  - see next slide for methods
- Get a `Query` object from entity manager:
  - `Query createQuery(String qlString);`
  - `<T> TypedQuery<T> createQuery(CriteriaQuery<T> criteriaQuery);`
  - `<T> TypedQuery<T> createQuery(String qlString, Class<T> resultClass);`
  - `Query createNamedQuery(String name);`
  - `<T> TypedQuery<T> createNamedQuery(String name, Class<T> resultClass);`
  - `Query createNativeQuery(String sqlString);`
  - `Query createNativeQuery(String sqlString, Class resultClass);`
  - `Query createNativeQuery(String sqlString, String resultSetMapping);`

Some `Query` Methods

- `List getResultList()`
- `Object getSingleResult()`
- `int executeUpdate()`
- `Query setMaxResults(int maxResult)`
- `Query setFirstResult(int startPosition)`
- `Query setHint(String hintName, Object value)`
- `Query setFlushMode(FlushModeType flushMode)`
- `Query setParameter(String name, Object value)`
- `Query setParameter(String name, Date value, TemporalType temporalType)`
- `Query setParameter(String name, Calendar value, TemporalType temporalType)`
- Plus `setParameter` methods with position instead of name

Example

- Note exceptions thrown by `getSingleResult()`

```
try{
   Query query = entityManager.createQuery(
      "from Employee e where e.name='Bill' ");
   Employee emp = (Employee)query.
                        getSingleResult();
} catch (EntityNotFoundException notFound) {
} catch (NonUniqueResultException nonUnique {
}
```

Example

- `getSingleResult` requires a query to return exactly one result
- `getResultList()` will return a list of zero or more results

```
Query query = entityManager.createQuery(
      "select e from Employee e
                        where e.name='Bill' ");
java.util.List emps = query.getResultList();
```

Parameters

- Use parameters in queries
  - As in JDBC prepared statements
  - Instead of creating a query with user-defined text
- Two ways
  - Named parameters use `:name` in the query and `setParameter("name", value)` to set it
  - Numbered parameters use `?n` in the query and `setParameter(n, value)` to set it
- Named parameters are preferable

Example

- Using `setParameter` to fill in named parameter

```
public List findByName(String name){
   Query query = entityManager.createQuery(
      "select e from Employee e
                        where e.name=:eName");
   query.setParameter("eName", name);
   return query.getResultList();
}
```

Date Parameters

- A Java `Date` or `Calendar` object can represent a date, time of day or timestamp
- Need to tell query object how to use these in setting corresponding parameters
  - recall `javax.persistence.TemporalType` has values `DATE, TIME, TIMESTAMP`
- Use `setParameter(String name, Date value, TemporalType type)` to set these
  - or overloaded one with `Calendar` instead of `Date`

Paging Results

- Query method `setMaxResults` and `setFirstResult` can be used to retrieve subset of query results
  - counting starts at 0
- Example:

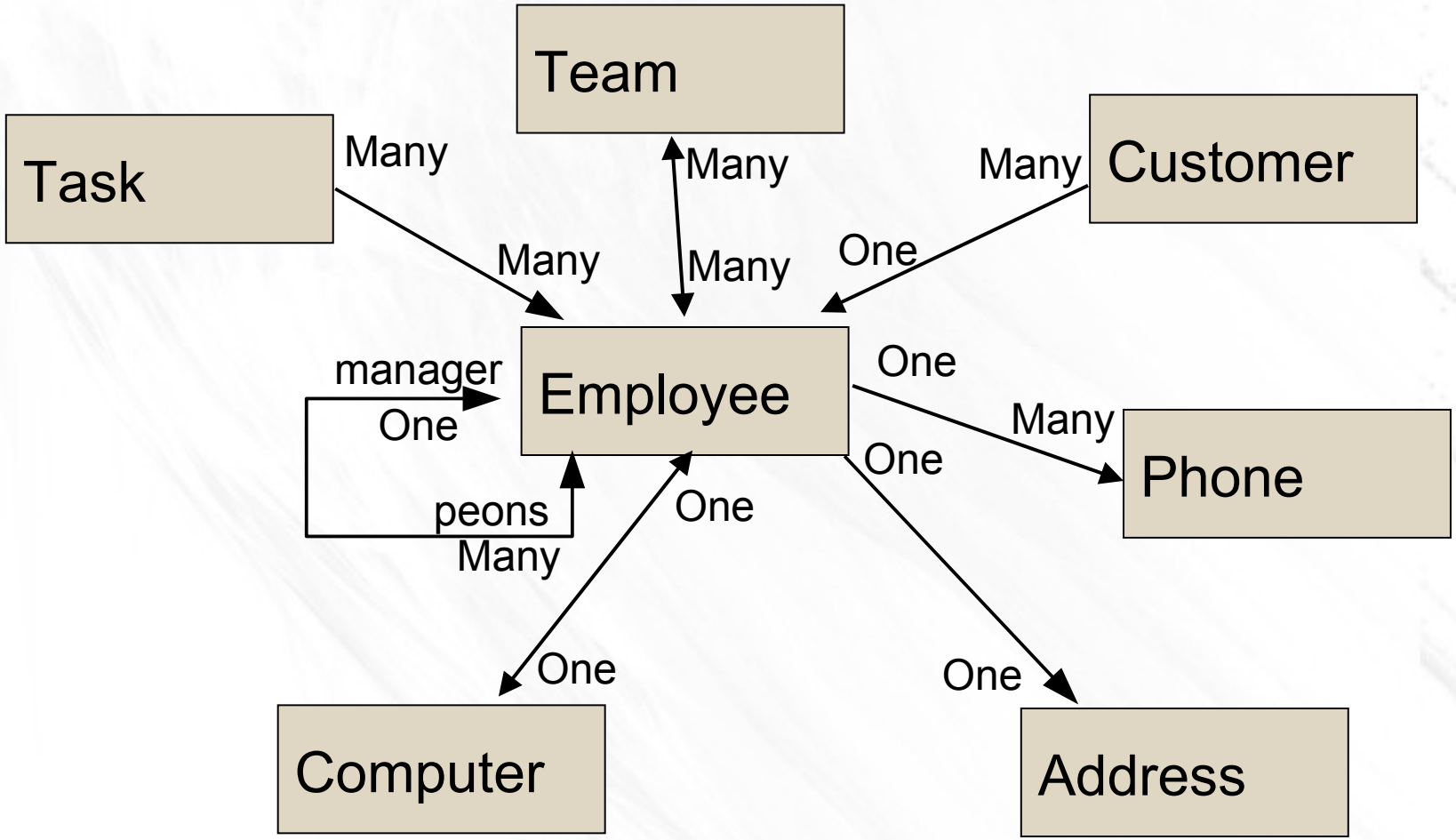
```
List results = query.setMaxResults(max).
               setFirstResult(index).
               getResultList();
```

- After query use `entityManager.clear()` to detach entries no longer in use

JPQL Examples

- Java Persistence queries use JPQL to define the query
  - JDBC uses SQL Table-based queries
  - JPQL uses entity-based queries
- Need to define JPQL which uses abstract persistence schema of entity beans
  - Abstract scheme name to identify beans
    - Defaults to entity bean class name
  - Persistence field to specify values
  - Relationship fields to navigate across relationships

Example Class Diagram



Simple Queries

- Select all Employee beans:
  - `SELECT OBJECT(e) FROM Employee AS e`
  - `SELECT e FROM Employee e`
  - `FROM`: which entity types included in select (abstract schema names)
  - `AS e`: assigns `e` to be identifier of `Employee` bean
    - `e` must be distinct from existing abstract-schema-names
    - Identification names not case sensitive
    - Identifiers cannot be JPA SQL reserved words
- `OBJECT()` operator is optional
  - when select type is solitary identifier for an entity

JPA QL Reserved Words

- Following are reserved (case insensitive):
  - ABS, ALL, AND, ANY, AS, ASC, AVG, BETWEEN, BIT_LENGTH, BOTH, BY, CASE, CHAR_LENGTH, CHARACTER_LENGTH, CLASS, COALESCE, CONCAT, COUNT, CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP, DELETE, DESC, DISTINCT, ELSE, EMPTY, END, ENTRY, ESCAPE, EXISTS, FALSE, FETCH, FROM, GROUP, HAVING, IN, INDEX, INNER, IS, JOIN, KEY, LEADING, LEFT, LENGTH, LIKE, LOCATE, LOWER, MAX, MEMBER, MIN, MOD, NEW, NOT, NULL, NULLIF, OBJECT, OF, OR, ORDER, OUTER, POSITION, SELECT, SET, SIZE, SOME, SQRT, SUBSTRING, SUM, THEN, TRAILING, TRIM, TRUE, TYPE, UNKNOWN, UPDATE, UPPER, VALUE, WHEN, WHERE.
- Good idea to also avoid SQL reserved words

Simple Queries with Paths

- `SELECT` clauses can return any basic or relationship properties
- Can use dot (.) to navigate
- Eg. return names of all employees
  - `final String query = "SELECT e.name FROM Employee AS e"`
  - `final List<String> names = em.createQuery(query).getResultList();`
- Eg. return addresses from all Employee beans
  - `SELECT e.address FROM Employee as e`
- Path can be as long as needed: a.b.c.d.e.f

Returning Multiple Columns

- SELECT clause can have multiple columns
  - Results returned in list of object array: `List<Object[]>`
- Example:
  - `final String query = "SELECT e.name, e.id FROM Employee AS e";`
  - `final List<Object[]> properties = em.createQuery(query).getResultList();`

Names in Queries

- Recall that we can make either an entity's fields or properties persistent
  - depending on where we use `@Id` and other annotations
- In general, a property name does not need to match a field name
- The names used in queries need to be the ones that are persisted for an entity
  - all field names or all property names

Rules for Paths

- Paths cannot navigate beyond persistent properties
  - If property value is non-entity JavaBean, path cannot access its instance fields
  - Unless JavaBean is annotated with `@Embeddable` and field is annotated `@Embedded`
- Path in SELECT clause must end in single type
  - not collection-based relationship field
- Path cannot navigate across a collection-based relationship field (use IN or JOIN operator)
  - `SELECT e.phones.number` is illegal

Constructor Expressions

- Can use constructor in a SELECT clause
  - To create a nonentity java object
  - Using specified columns for the constructor parameters
- E.g. suppose have defined Name
  - With a 2 argument constructor `Name(Long id, String name)`
- Can call constructor in query:
  - `SELECT NEW org.mypackage.Name(e.id, e.name) FROM Employee e`
  - Query object will allocate new Name for each row returned

`IN` Operator

- Allows identifier to represent individual elements in a collection-based relationship
  - Eg. return all phones of all employees
    - `SELECT p FROM Employee AS e, IN(e.phones) AS p`
- With identifier to represent individual elements of collection can use in further expressions
  - Eg. return all phone numbers for which employees have in some phone
    - `SELECT p.number FROM Employee AS e, IN(e.phones) AS p`
- Only includes Employees that have some phones

INNER JOIN

- Alternative syntax for `IN`, similar to SQL
- `SELECT p.number FROM Employee e INNER JOIN e.phones p`
  - Same as `SELECT p.number FROM Employee e, IN(e.phones) p`
- Reserved word `INNER` is optional
  - E.g find phone numbers for all employees who have a task
    - `SELECT p.number FROM Task t JOIN t.owners o JOIN o.phones p`
    - `SELECT p.number FROM Task t, IN(t.owners) o, IN(o.phones) p`

LEFT JOIN

- Retrieve a set of entities where matching values in join statement may not exist
  - null values placed in result set for missing values
- E.g. retrieve names and phone numbers (which may be null)
  - `SELECT e.name, p.number FROM Employee e LEFT JOIN e.phones p`
- Can also express as `LEFT OUTER JOIN`
  - To match SQL-92

Fetch Joins

- `JOIN FETCH` syntax allows preloading an entity's relationships even if they have `FetchType.LAZY`
- Reduces data base queries required
- E.g. suppose phones are lazy fetched
  - `SELECT e FROM Employee e LEFT JOIN FETCH e.phones`
    - Gets all employees with/without phone
  - Employees returned would have phone field prefetched
  - Uses one query instead of N+1 queries

Using `DISTINCT`

- `DISTINCT` keyword ensures query does not return duplicates
  - List query return type allows duplicates
- E.g. return all distinct employees belonging to a team
  - `SELECT DISTINCT emp FROM Team t JOIN t.members emp`

`WHERE` Clause

- Similar to WHERE clause in SQL, limits the elements selected
  - Lots of details – see text
- Can use path expressions in same way as in `SELECT` clause
- Comparisons with literal:
  - Path must end in persistence field/property
  - Can use literal strings (strings use single quotes, double to escape single quote in a string), long, double, boolean (TRUE, FALSE), named or positional parameters
  - E.g. `SELECT e FROM Employee e WHERE e.name = 'Fred Bloggs'`

`WHERE` Clause Operator Precedence

- Expressions can use the following operators (from highest to lowest precedence):
  - Navigation (.)
  - Arithmethic: unary +, - ; multiplicative *, /; additive + -
  - Comparison: =, >, >=, <, <=, <>, `LIKE`, `BETWEEN`, `IN`, `IS NULL`, `IS EMPTY`, `MEMBER OF`
  - Logical: `NOT`, `AND`, `OR`
- Details follow

`WHERE` Clause and Arithmetic Operators

- Arithmetic operators only used in `WHERE` clause, not `SELECT` clause
- Rules same as in Java, including promotion rules
- `String`, `boolean`, entity object types *cannot* be used in arithmetic operations
  - function for string concatenation described below
- Example:
  - `SELECT e FROM Employee e WHERE (e.id * .01) > 300.00`

`WHERE` Clause and Logical Operators

- `AND`, `OR`, `NOT` operate the same way in JPQL as in SQL
  - Not as in Java
- Operands must be `TRUE`, `FALSE`, or `NULL`
- Lowest Precedence
- JPQL does not specify whether both sides are evaluated or short circuiting is used
  - Depends on native query language used underneath

SQL Truth Tables

- Operations involving `NULL` return *unknown*
- If `AND` and `OR` do not short circuit:

p	*q*	*p* OR *q*	*p* AND *q*	*p* = *q*
True	True	True	True	True
True	False	True	False	False
True	Unknown	True	Unknown	Unknown
False	True	True	False	False
False	False	False	False	True
False	Unknown	Unknown	False	Unknown
Unknown	True	True	Unknown	Unknown
Unknown	False	Unknown	False	Unknown
Unknown	Unknown	Unknown	Unknown	Unknown

p	NOT *p*
True	False
False	True
Unknown	Unknown

`WHERE` Clause and Comparison

- Only = and <> may be used on boolean and Entity object identifiers
- All relational operators can be used on numbers
- Can use <, <=, >, >= on strings
  - But standard does not define results
  - See database documentation for specific database
    - If you use, you may be stuck with the database in the future

`WHERE` Clause and Equality

- Numerical values can be compared with promotion rules applying
  - Primitive types can be compared with wrapper types
- Strings can be compared for equality
  - Results depend on underlying database
- Entity objects must be same type from same deployment
  - Equality means equal primary keys
- Can use Date objects in equality operations

`WHERE` Clause and `BETWEEN`

- `BETWEEN` indicates *inclusive* range of values
  - e.g. `SELECT e FROM Employee e WHERE e.id BETWEEN 100 AND 200`
- Values may be numeric primitives or wrapper types
  - Cannot be used on `String`, `boolean` or entity object references
- `NOT BETWEEN` specifies complement
  - Strictly outside range
  - E.g. `SELECT e FROM Employee e WHERE e.id NOT BETWEEN 100 AND 200`
  - Same as `SELECT e FROM Employee e WHERE e.id < 100 OR e.id > 200`

`WHERE` Clause and `IN`

- `IN` used in `WHERE` clause is different than in `FROM` clause
- Here, `IN` tests for membership in list of literal values
- Values can be string, numeric, as well as input parameters
  - Eg. `SELECT e FROM Employee e WHERE e.address.state IN (:state1, :state2, :state3, 'BC', 'AB')`
- If field is `NULL` value of expression is *unknown*
- `NOT IN` reverses selection

`WHERE` Clause and `IS NULL`

- Allows checking whether path expression is *null*
- E.g. return employees without a manager
  - `SELECT e FROM Employee AS e WHERE e.manager IS NULL`
- Note: path expressions are composed with inner join semantics
  - null fields in path eliminate entity from consideration
- `NOT NULL` can be used
  - To test input parameters
  - To eliminate null values in query result
- Read carefully the text section on `IS NULL`

`WHERE` Clause and `IS EMPTY`

- `IS EMPTY` (`IS NOT EMPTY`) tests whether a collection-based relationship is (not) empty
  - Recall it will never be NULL!
  - E.g. `SELECT e FROM Employee e WHERE e.phones IS EMPTY`
- Not allowed to use on an identifier associated with an `IN` operator in a `FROM` clause
  - e.g. `IN(e.phones) AS p`
    - Cannot say `p IS NOT EMPTY`
    - Because `IN` clause excludes those employees that have empty phones field

`WHERE` Clause and `MEMBER OF`

- `MEMBER OF` determines whether an entity is a member of a collection-based relationship
- E.g. Find teams with a given employee as member
  - `SELECT t FROM Team AS t, IN(t.members) AS m, Employee AS e WHERE e = :myEmployee AND e MEMBER OF t.members`

`WHERE` Clause and `LIKE`

- `LIKE` allows matching a pattern
- Two special characters in pattern
  - `%` stands for any sequence of characters
  - `_` stands for any single character
  - Escape them with `\`
- Input parameters can be used to generate pattern
  - `SELECT e FROM Employee e WHERE e.name LIKE :pattern`

Functional Expressions in `WHERE` Clause

- `LOWER(string)` - convert to lower case
- `UPPER(string)` - convert to upper case
- `TRIM([[LEADING | TRAILING | BOTH] [trim_char] FROM] string)`
  - If no trim char specified, trims spaces
- `CONCAT(string1, string2)`
- `LENGTH(string)`
- `LOCATE(string, substr [, start])`
  - 0 if not found, positions start at 1
- `SUBSTRING(string, start, length)`

Functional Expressions in `WHERE` Clause

- Arithmetic functions can be applied to primitives and wrapper types
  - `ABS(number)`
  - `SQRT(double)`
  - `MOD(int, int)`
- Functions returning date and time
  - `CURRENT_DATE`
  - `CURRENT_TIME`
  - `CURRENT_TIMESTAMP`

Aggregate Functions in `SELECT` Clause

- `COUNT(`identifier or path expression`)`
- `MAX(`path expression`)`, `MIN(`path expression`)`
- `AVG(`numeric path expression`)`, `SUM(`numeric path expression`)`
- `DISTINCT` eliminates duplicate values
  - `SELECT DISTINCT COUNT(e.address.zip) FROM Employee e WHERE e.address.zip LIKE '554%'`
- Fields with *null* value eliminated from result set operated on by aggregate functions
- `COUNT` returns 0 when evaluating empty collection
- `AVG`, `MIN`, `MAX`, `SUM` return *null* for empty collection

`ORDER BY` Clause

- Allows you to specify the order of entities in the collection returned by a query
  - field used in `ORDER BY` clause must be in `SELECT` clause
- Similar to SQL
- Can use `ASC`, `DESC`
  - Default is `ASC`
- E.g. `SELECT e FROM Employee AS e ORDER BY e.name ASC`

`GROUP BY` and `HAVING`

- `GROUP BY` allows grouping data for aggregate functions
  - same as in SQL
  - must specify column from the `SELECT` clause
- `HAVING` restricts groups to ones satisfying a boolean condition
  - E.g. Names of employees that manage more than 10 peons with a count of the number they manage:
  - `SELECT e.name, COUNT(p) FROM Employee e JOIN e.peons p GROUP BY e.name HAVING COUNT(p) > 10`

Subqueries

- Subqueries are `SELECT` statements embedded in another query
- JPQL supports subqueries in `WHERE` and `HAVING` clauses
  - e.g. `SELECT COUNT(res) FROM Reservation res WHERE res.amountPaid > (SELECT AVG(r.amountPaid) FROM Reservation r)`
- Subquery can reference identifiers in `FROM` clause of outer query
  - e.g. `SELECT cr FROM Cruise cr WHERE 100000 < (SELECT SUM(res.amountPaid) FROM cr.reservations res)`

Subquery Modifiers: `ALL, ANY, SOME, EXISTS`

- These are quantifiers in front of the subquery
- `ALL` returns true if *all* the things in the subquery match the conditional expression
  - e.g. `FROM Cruise cr WHERE 0 < ALL (SELECT res.amountPaid FROM cr.reservations res)`
- `SOME` (or `ANY`) returns true if *anything* in the subquery matches the conditional expression
- `EXISTS` returns true if the subquery returns one or more values
  - e.g. `SELECT cr FROM Cruise cr WHERE EXISTS (SELECT res FROM cr.reservations WHERE res.amountPaid = 0)`

Bulk `UPDATE` and `DELETE`

- JPQL has `UPDATE` and `DELETE` similar to SQL
  - e.g. `UPDATE Reservation res SET res.amountPaid = (res.amountPaid + 10) WHERE EXISTS (SELECT c FROM res.customers c WHERE c.firstName = 'Bill' AND c.lastName = 'Burke')`
- Note: implementations required to update database appropriately, *not* update managed entities

Native Queries

- Sometimes you want to use vendor-specific proprietary SQL extensions
- Entity Manager provides a way to create native SQL queries and map to your objects
  - Queries can return entities, column values, or a mixture
  - EntityManager interface has three methods dealing with these three cases
- Can also inject datasource using @Resource and use JDBC
  - Changes not reflected in persistence context

Scalar Native Queries

- `Query createNativeQuery(String sql)`
- Creates a native query that return scalar results
  - Native SQL is passed as parameter
- Query executes sql and returns the result set in format used by JPQL for scalar values
  - Each row is `Object` array
  - `getResultList()` returns list of `Object[]`

Simple Entity Native Queries

- `Query createNativeQuery(String sql, Class entityClass)`
- Takes SQL statement and maps to one entity
  - Based on mapping metadata defined for the entity
- Columns returned must match entity's O/R mapping
- All entity properties must be selected

Complex Native Queries

- `Query createNativeQuery(String sql, String mappingName)`
- `mappingName` matches a declared `@javax.persistence.SqlResultSetMapping`
- Mapping defines how native SQL results fit back into O/R model
- See text for example
  - Not for faint hearted

Named Queries

- Can predefine JPQL or native SQL queries and reference them by name in a query
- Same reason you declare constants - to reuse common queries and define them in one place
- Use `jakarta.persistence.NamedQuery` annotation
  - Next slide

`@NamedQuery`

- Three attributes
  - `String name()` - name of query
  - `String query()` - JPQL
  - `QueryHint[] hints() default {}` - list of vendor specific parameters
- `@QueryHint` has two attributes
  - `String name()`
  - `String value()`
- There is also `@NamedQueries` which takes a list of `@NamedQuery` in `{}`

Example Named Queries

```
@NamedQueries({
   @NamedQuery(name="getAverageReservation",
        query = "SELECT AVG(r.amountPaid) "
   + "FROM Cruise AS c, JOIN c.reservations r "
   + "WHERE c = :cruise"),
   @NamedQuery(name="findFullyPaidCruises",
    query="SELECT cr FROM Cruise cr WHERE 0 < "
          + "ALL(SELECT res.amountPaid "
          + "FROM cr.reservations res)")
  })
```

Summary

- This concludes the core of using entities
- There is more to the story but this is the hardest part
- Database experience transfers to knowing the sort of query you want to generate
- Be sure to test your queries, especially considering the effects of nulls
  - Test, test, test, test, oops, test